LD8-S1 Exact Periodic Support-Atlas Specification

Packed global CIC sources, padded-bitset dilation, transactional planning, and equivalence

mdstats development specification

2026-07-21

Contents

LD8-S1 - Exact periodic support atlas

Package target: `mdstats 0.19.55a0`
Status: implemented
Primary modules:

```
mdstats.plotting.density_support_atlas
mdstats.plotting.density_scene_planning
```

Purpose

LD8-S1 constructs the exact finite support of one scientific density channel without allocating the complete source-node by Gaussian-stencil pair array. It deposits all selected samples once into one global periodic CIC source, packs that source by storage block, and computes the exact modular support with bounded source-block bitset dilation.

The scientific estimator remains unchanged. S1 does not yet compute density values; target-owned direct realization begins in LD8-S2. LD7 remains the production fallback.

Mathematical support contract

Let

- $\mathbf{N} = (N_x, N_y, N_z)$ be the logical grid;
- $\mathcal{S}$ be the set of positive global CIC source nodes;
- $\mathcal{K}_\varepsilon$ be the exact retained canonical stencil offsets at $\varepsilon = 10^{-8}$.

The required support is the periodic modular Minkowski sum

$$\mathcal{A} = \{(\mathbf{n}_s + \boldsymbol{\delta}) \bmod \mathbf{N} : \mathbf{n}_s \in \mathcal{S}, \boldsymbol{\delta} \in \mathcal{K}_\varepsilon\}.$$

Every node in $\mathcal{A}$ is represented exactly once in the atlas. Every node outside $\mathcal{A}$ is an exact implicit zero.

Packed global CIC source

```
@dataclass(frozen=True, slots=True)
class PeriodicPackedCICSourceField3D:
    logical_grid_shape: tuple[int, int, int]
    storage_block_shape: tuple[int, int, int]
    source_block_indices: NDArray[np.int32]
    occupancy_bitsets: NDArray[np.uint64]
    block_value_offsets: NDArray[np.int64]
    packed_values: NDArray[np.float64]
    total_measure: float
    source_provenance: DensitySourceProvenance
    metadata: FrozenJSONMapping
```

Schema:

```
mdstats.periodic-packed-cic-source-field.v1
```

The adapter

```
pack_periodic_cic_source(
    source: SparseCICNodeMasses3D,
    *,
    storage_block_shape: tuple[int, int, int] = (16, 16, 16),
    ...
) -> PeriodicPackedCICSourceField3D
```

preserves all positive source-node masses and their canonical global flat indices. Values in each block follow increasing local C-order bit index. Source blocks follow increasing global C-order block index.

The invariant is

$$\sum_i m_i = M_{\text{target}}$$

within the existing CIC conservation tolerance. Every packed value is strictly positive.

Support-atlas record

```
@dataclass(frozen=True, slots=True)
class DensitySupportAtlas:
    logical_grid_shape: tuple[int, int, int]
    storage_block_shape: tuple[int, int, int]
    source_field_identity: str
    routing_identity: str
    active_target_block_indices: NDArray[np.int32]
    target_support_bitsets: NDArray[np.uint64]
    source_to_target_block_ranges: NDArray[np.int64]
    source_to_target_block_indices: NDArray[np.int32]
    connected_component_labels: NDArray[np.int32] | None
    planning: DensitySupportAtlasPlan
    metadata: FrozenJSONMapping
```

Schema:

```
mdstats.density-support-atlas.v1
```

The source-to-target arrays form CSR ownership metadata. For source block i, its target-block rows occupy

```
source_to_target_block_indices[
    source_to_target_block_ranges[i]:
    source_to_target_block_ranges[i + 1]
]
```

The atlas is field-specific. Its identity includes the full packed-source identity and the routing identity. It is not placed in a cross-field global cache.

Exact padded-bitset dilation

For one source block, let its valid extent be

$$\mathbf{E} = (E_x, E_y, E_z),$$

and define componentwise stencil extrema

$$\boldsymbol{\delta}_{\min}, \qquad \boldsymbol{\delta}_{\max}.$$

The lifted brick shape is

$$\mathbf{Q} = \mathbf{E} + \boldsymbol{\delta}_{\max} - \boldsymbol{\delta}_{\min}.$$

Each occupied local source coordinate $\mathbf{u}$ is embedded at

$$\mathbf{q}_s = \mathbf{u} - \boldsymbol{\delta}_{\min}.$$

For every stencil offset $\boldsymbol{\delta}$, its target coordinate is

$$\mathbf{q}_t = \mathbf{q}_s + \boldsymbol{\delta}.$$

By construction,

$$0 \le q_{t,a} < Q_a$$

for every axis, so C-order integer bit shifts are exact and cannot wrap between lifted-brick rows. The implementation:

1. packs the occupied embedded source nodes into one Python integer;
2. applies one signed C-order bit shift per retained stencil offset;
3. bitwise-ORs the shifted integers;
4. unpacks the resulting lifted support once;
5. maps lifted coordinates back to global logical coordinates;
6. folds those coordinates modulo $\mathbf{N}$;
7. packs them into canonical target-block bitsets.

The algorithm identifier is:

`ld8_s1_exact_source_block_padded_bitset_dilation_v1`

It performs

$$N_{\text{source blocks}}\,|\mathcal{K}_\varepsilon|$$

bitset shifts instead of enumerating

$$N_{\text{source nodes}}\,|\mathcal{K}_\varepsilon|$$

fine interactions. This comparison is an operation-count reference; one large-integer shift is not assumed to cost the same as one scalar pair operation.

Correctness argument

For a fixed source block, every positive source node is embedded injectively. For every exact signed stencil offset, the lifted shift maps that source bit to the unique lifted coordinate corresponding to ordinary integer addition. The halo bounds guarantee no flattened-coordinate carry error. Folding the resulting global coordinate modulo $\mathbf{N}$ is exactly the periodic action in the support definition.

OR reduction removes duplicates without removing any reachable node. Packing by canonical target block preserves the set. Therefore the union over all source blocks is exactly $\mathcal{A}$.

Small-grid audit code may explicitly enumerate the modular Minkowski sum and compare sorted flat indices. That verifier is bounded and never used for production-sized planning.

Transactional planner

```
@dataclass(frozen=True, slots=True)
class DensitySupportPlanningLimits:
    max_target_blocks: int = 1_000_000
    max_source_target_edges: int = 20_000_000
    max_bitset_region_operations: int = 250_000_000
    max_retained_bytes: int = 1_000_000_000
    max_transient_bytes: int = 1_000_000_000

plan_density_support_atlas(
    source_field: PeriodicPackedCICSourceField3D,
    routing: PeriodicKernelBlockRouting,
    *,
    limits: DensitySupportPlanningLimits | None = None,
) -> DensitySupportAtlasPlan
```

Planning records:

- source node and source-block counts;
- stencil-offset and block-word counts;
- target-block and source-target-edge upper bounds;
- target support-node upper bound;
- exact bitset-shift operation count;
- complete fine-pair count for reference only;
- source, routing, and atlas retained-byte bounds;
- maximum lifted-brick nodes;
- conservative maximum lifted transient bytes;
- total predicted peak bytes;
- explicit failing limit names.

The lifted transient estimate includes packed integers, unpacked masks, coordinate arrays, sorting arrays, source-local arrays, and stencil-shift arrays. Only one lifted source-block brick is live at a time.

Planning raises `GraphComplexityError` before target atlas allocation when any hard limit fails.

Connected components

Periodic face-connected target-block components are optional and lazy:

```
build_density_support_atlas(
    source_field,
    routing,
    *,
    compute_connected_components=False,
)
```

When requested, component labels use six face neighbors with periodic block-lattice wrap. Components are support-planning metadata only; they do not alter target masks.

Serialization and identities

Every public record has canonical JSON serialization and round-trip construction. Scientific arrays are immutable and C-contiguous. SHA-256 content identities include all arrays that affect support.

The support atlas metadata must state:

```
complete_fine_pair_array_allocated = false
source_specific_global_cache_used = false
```

Full-trajectory evidence

The implementation was run on all 1,500 frames and the four production channels at `kernel_tail_tolerance=1e-8`:

Channel	Grid	Source nodes	Source blocks	Target blocks	Exact target nodes	Atlas time	Pair/shift ratio	Atlas bytes
Na	540^3	36,280	280	1,322	1,728,706	13.388 s	129.6x	0.685 MiB
Si	1038^3	54,680	280	1,381	1,833,591	13.301 s	195.3x	0.714 MiB
Al	1037^3	57,471	308	1,432	1,952,525	15.195 s	186.6x	0.742 MiB
O	646^3	187,821	1,132	5,625	7,274,190	55.158 s	165.9x	2.915 MiB

Every target-node count exactly matches the completed LD7 `$10^{-8}$` baseline. The complete benchmark required 138.680 s including repeated resolution analysis and source preparation. Atlas construction alone required 97.042 s in aggregate.

These measurements validate support correctness and bounded storage. They do not yet satisfy the final LD8 scientific-time target because density-value realization remains LD7 until S2/S3.

Failure conditions

Construction fails explicitly when:

- source and routing grid or block shapes differ;
- source blocks or local source bits are invalid;
- source measure is nonpositive or not conserved;
- source or routing identities are malformed;
- planner limits fail;
- terminal invalid bits appear in target masks;
- target blocks are unsorted or duplicated;
- source-to-target CSR offsets are inconsistent;
- optional components are misaligned;
- an explicit equivalence audit finds a missing or extra node.

Focused validation

Tests include:

- exact support against explicit modular Minkowski sums;
- divisible and partial terminal grids;
- periodic faces, edges, and corners;
- overlapping source supports and duplicate suppression;
- deterministic randomized terminal-grid cases;
- packed-source conservation and JSON identity;
- atlas JSON identity and CSR consistency;
- resource preflight rejection;
- exact production target-node-count agreement for benchmark evidence.

Scope exclusions

LD8-S1 does not:

- calculate Gaussian-weighted density values;
- select direct versus FFT execution;
- replace the production LD7 backend;
- normalize or pack a realized target density;
- change HDR selection;
- change rendering.

Those functions belong to LD8-S2 through S4 and LD9.

External attribution

Periodic CIC deposition follows Hockney and Eastwood, *Computer Simulation Using Particles* (1988). The finite normalized Gaussian stencil is governed by the existing kernel specification. The source-block padded-bitset dilation, terminal handling, exact support atlas, and planning model are mdstats-specific designs.